



Problem statements

Note: Candidates may choose one (1) problem statement.

Problem statement 1

Theme : Multimodal AI: Audio search

Problem statement name : *Effective retrieval from audio transcripts*

Overview

You have audio recordings across conversations, each containing two speakers. How might you achieve effective hybrid search (e.g. both keyword + semantic search) across them? What diarization, database, embeddings, and indexing strategies might result in “ideal” retrieval quality at scale? Moreover: What metrics are important to consider should this system be brought to production, and what might constitute an effective evaluation of the system?

Tech stack

- Python or Typescript preferred
- Some RDBS with embedding support such as Postgres and pgvector

Restrictions

- A coding agent may be used, but you must explain how you collaborated with (prompted) the agent to arrive at your solution.

Task

1. Construct an effective golden dataset of 5-6 audio files of 8-10 minutes length. Each should have a unique set of two speakers. You may use recordings of interviews, or podcasts.
2. Decide what you feel an effective hybrid search strategy might be across these audio files. Users should be able to search for both specific words uttered, as well as terms semantically similar to their query.
3. Implement a hybrid search solution across them using a locally run solution. Transcripts should be generated from each audio file. Searching for a term should return results highlighting the containing file, timestamp, and speaker. Embedding generation and indexing should run locally; transcription may use a hosted or local solution.
4. Evaluate the success of your solution by creating automated tests that measure the recall@k of your solution against a small labeled query set.



Submission format

- Details about your solution, including an explanation of your engineering design, your rationale for it, definition of success criteria for your solution, level of achievement against your criteria, and any limitations of your solution, in a Markdown or PDF file
- Disclosure of coding agent use: You must disclose the use of coding agents in developing your project. Where used, you must explain how you directed the agent to arrive at the outputs it generated. This may be achieved by sharing documentation of the architectural requirements you supplied to the agent, or by providing summarized traces of your interactions with the agent.
- GitHub/GitLab for code commit containing your solution, as well as your golden dataset and tests

Problem statement 2

Theme : Multimodal AI: Images

Problem statement name : *Enrichment of image generation using structured context*

Overview

Consider an image generation pipeline which depends on structured context to refine the outputs. How would you guarantee that generation quality is achieved in the generation process?

Note: This exercise requires an API key for accessing Gemini models. You can request a free API key for yourself at <https://aistudio.google.com/>

Tech stack

- Python, Typescript, or Ruby preferred

Task

1. Propose an image generation pipeline for display advertisement generation that uses Gemini 3.1 Flash-Lite Image or Gemini 3.1 Flash Image. The pipeline should take a reference product image plus three structured text fields (target geography, season, and freeform text that must be included in the generated image) as input. This added context leads to refinement of the generated image. *Note: Text rendering may not reliably be perfect; this is acceptable, but consider this in your solution.*
2. Decide an effective generation strategy for images generated using this pipeline, then implement the generation pipeline. The maximum resolution of generated images must be 1K (long edge ≤ 1024 px).
3. Implement an evaluator that judges the quality of the outputs across ≈ 20 images output from the pipeline.
4. Demonstrate that your evaluator can identify passing and failing outputs against quality metrics you define through automated tests. At a minimum, consider 1) context adherence, 2) fidelity of reference product, and 3) text rendering fidelity.

Restrictions



- A coding agent may be used, but you must explain how you collaborated with (prompted) the agent to arrive at your solution.

Submission format

- Details about your solution, including an explanation of your engineering design, your rationale for it, definition of success criteria for your solution, level of achievement against your criteria, and any limitations of your solution, in a Markdown or PDF file
- Disclosure of coding agent use: You must disclose the use of coding agents in developing your project. Where used, you must explain how you directed the agent to arrive at the outputs it generated. This may be achieved by sharing documentation of the architectural requirements you supplied to the agent, or by providing summarized traces of your interactions with the agent.
- GitHub/GitLab for code commit containing your solution, as well as your golden dataset and tests

Problem statement 3

Theme : User generated content

Problem statement name : *Rewarding novelty in submissions*

Overview

You are faced with judging the “novelty” of some user generated text-based content submitted in response to some larger piece of content.. How would you create a pipeline that rewards truly novel content on a normalized scale of [0.0,1.0]?

Note: Depending on your approach, this exercise might benefit from access to Gemini models. You can request a free API key for yourself at <https://aistudio.google.com/>

Tech stack

- Python, Typescript, or Ruby preferred

Task

1. Determine the particular “shape” of user generated content you would evaluate. “User generated content” means structured input a user might submit to an app or website in response to some fixed piece of content e.g., commentary on a news article. *Note: The structured content must contain three discrete properties provided by the user (e.g. a headline, body, and a multi-choice selection); the fixed content that the user content is submitted against should be 100 words or less.*
2. Describe how you would reward submitted user generated content based on its novelty relative to other submissions of its kind, while retaining a minimum level of relevancy to the fixed content in question.
3. Implement a mechanism to evaluate the novelty of a newly submitted piece of content against ≈50 other pieces of content. *Note: synthetic generation of such content via an LLM is recommended.*



4. Demonstrate how you effectively reward truly novel content, do not reward non-“novel” content, and remain relevant to the fixed content through automated tests. You must show high novelty, but low relevance submissions are not rewarded.

Restrictions

- A coding agent may be used, but you must explain how you collaborated with (prompted) the agent to arrive at your solution.

Submission format

- Details about your solution, including an explanation of your engineering design, your rationale for it, definition of success criteria for your solution, and level of achievement against your criteria, in a Markdown or PDF file
- Disclosure of coding agent use: You must disclose the use of coding agents in developing your project. Where used, you must explain how you directed the agent to arrive at the outputs it generated. This may be achieved by sharing documentation of the architectural requirements you supplied to the agent, or by providing summarized traces of your interactions with the agent.
- GitHub/GitLab for code commit containing your solution, as well as your golden dataset and tests

